

BugBoard26

Documento di Specifica dei Requisiti Software

Progetto di Ingegneria del Software
A.A. 2025/2026

Fabrizio Padulano Matricola: N86005058

Federico Padulano Matricola: N85005043

Indice

1	Introduzione	4
1.1	Scopo	4
1.2	Scope	4
1.2.1	Incluso	4
2	Glossario	4
3	Modellazione dei Casi d'Uso	6
3.1	Autenticazione Utente (Requisito 1)	6
3.2	Segnalare Issue (Requisito 2)	6
3.3	Visualizzare Vista Riepilogativa Issue (Requisito 3)	7
3.4	Assegnare Bug (Requisito 4)	8
3.5	Modificare Bug Assegnato (Requisito 9)	9
3.6	Impostare Scadenza Issue (Requisito 18)	10
4	Individuazione e Caratterizzazione degli Utenti (Attori) del Sistema	11
5	Requisiti Non-Funzionali e di Sistema	15
5.1	Sicurezza	15
5.1.1	1. Autenticazione e Crittografia	15
5.1.2	2. Controllo Accessi	15
5.2	Performance	15
5.2.1	3. Tempi di Risposta	15
5.2.2	4. Caricamento e Filtraggio Veloce	15
5.3	Affidabilità e Disponibilità	15
5.3.1	5. Gestione Errori	15
5.4	Usabilità	15
5.4.1	6. Interfaccia Intuitiva	15
5.4.2	7. Design Responsive	16
5.5	Architettura	16
5.5.1	8. Separazione Front-end e Back-end	16
5.5.2	9. Containerizzazione e Deployment	16
5.6	Vincoli di Implementazione e Sviluppo	16
5.6.1	10. Linguaggi e Paradigmi	16
5.6.2	11. Testing e Qualità del Codice	16
5.6.3	12. Indipendenza della Logica (Divieto MBaaS esclusivo)	16
5.6.4	13. Strumenti di Progettazione	16
6	Formalizzazione Caso d'Uso Significativo	17
7	Documento di Design del sistema	21
7.1	Analisi dell'architettura	21
7.1.1	Client (Front-end)	21
7.1.2	Server (Back-end)	22
7.2	Architetture di Riferimento	23
7.2.1	Architettura a 3 Livelli (3-Tier) e Strati Chiusi	23
7.2.2	Architettura a Repository (Repository Pattern)	23

7.3	Scelte tecnologiche adottate	23
7.3.1	Core Technologies	23
7.3.2	Infrastruttura e Deployment	24
8	Documento di Design del Software, documentazione del processo di sviluppo, e artefatti software.	24
8.1	Schema per la persistenza dati	24
8.2	Diagramma delle classi di design	25
8.2.1	Modulo Gestione Issue (Core Business)	26
8.2.2	Modulo Notifiche	28
8.3	Motivazioni delle Scelte dei Design Pattern	29
8.4	Gestione del Codice e Versioning	29
8.4.1	Repository GitHub	30
8.4.2	Statistiche e Flusso di Lavoro	30
8.5	Qualità del Codice	30
8.5.1	Analisi dei Risultati	31
8.6	DockerFile e file di build Automatica	33
8.6.1	Backend Dockerfile	33
8.6.2	Frontend Dockerfile	33
8.6.3	Orchestrazione con Docker Compose	34
9	Attività di Testing	36
9.1	Test Plan Funzionale	36
9.2	Test di Unità Automatici	36
9.3	Strategie Black Box: Analisi del Dominio di Input	39
9.3.1	Analisi del Test 1: Creazione Issue (IssueService)	39
9.3.2	Analisi del Test 2: Delega agli Adapter (NotificationService)	40
9.4	Strategie Strutturali e Isolation Testing	41
9.4.1	Copertura Strutturale (Statement Coverage)	41
9.4.2	Isolation Testing con Mock Objects	41
9.4.3	Conclusioni sulla Strategia	41

1 Introduzione

1.1 Scopo

Questo documento specifica i requisiti funzionali e non-funzionali del sistema BugBoard26, una piattaforma web per la gestione collaborativa di issue in progetti software.

1.2 Scope

1.2.1 Incluso

- Autenticazione e gestione utenti (2 ruoli: admin, normale)
- Creazione, modifica e visualizzazione di issue
- Assegnazione di bug ai membri del team
- Notifiche automatiche
- Filtri e ricerca issue
- API REST back-end
- Interfaccia web responsive

2 Glossario

Il glossario definisce i termini chiave del dominio applicativo utilizzati nel sistema BugBoard26.

Bug Errore nel software da sistemare.

Stato Posizione della issue: da fare, in lavorazione o risolta.

Priorità Quanto è urgente la issue.

Amministratore Chi gestisce il sistema, assegna i bug e crea account.

Utente normale Chi segnala issue e lavora su quelle assegnate.

Assegnazione Dare un bug a una persona specifica del team.

Etichetta Tag per categorizzare le issue (es. "frontend", "urgente").

Archiviazione Spostare una issue fuori dalle liste principali.

Bug duplicato Problema già segnalato. Viene chiuso.

Carico di lavoro Quanti bug ha una persona da sistemare.

Cronologia Storico di tutti i cambiamenti fatti su una issue.

Dashboard Pagina per amministratori con issues.

Tempo medio di risoluzione Quanto tempo ci vuole in media per sistemare un bug.

Scadenza Data entro cui si dovrebbe risolvere la issue.

Notifica Messaggio automatico per avvisare di cose importanti.

3 Modellazione dei Casi d'Uso

3.1 Autenticazione Utente (Requisito 1)

Deve essere implementato un sistema di autenticazione semplice e sicuro, basato su email e password. Le informazioni gestite dall'applicazione sono critiche per l'azienda, ed è fondamentale preservarne l'integrità e la segretezza. Il sistema viene fornito con un account da amministratore già attivo, con credenziali di default. Un amministratore può creare ulteriori utenze, specificando una email, una password, e indicando se quell'utenza sarà "normale" oppure "di amministrazione".

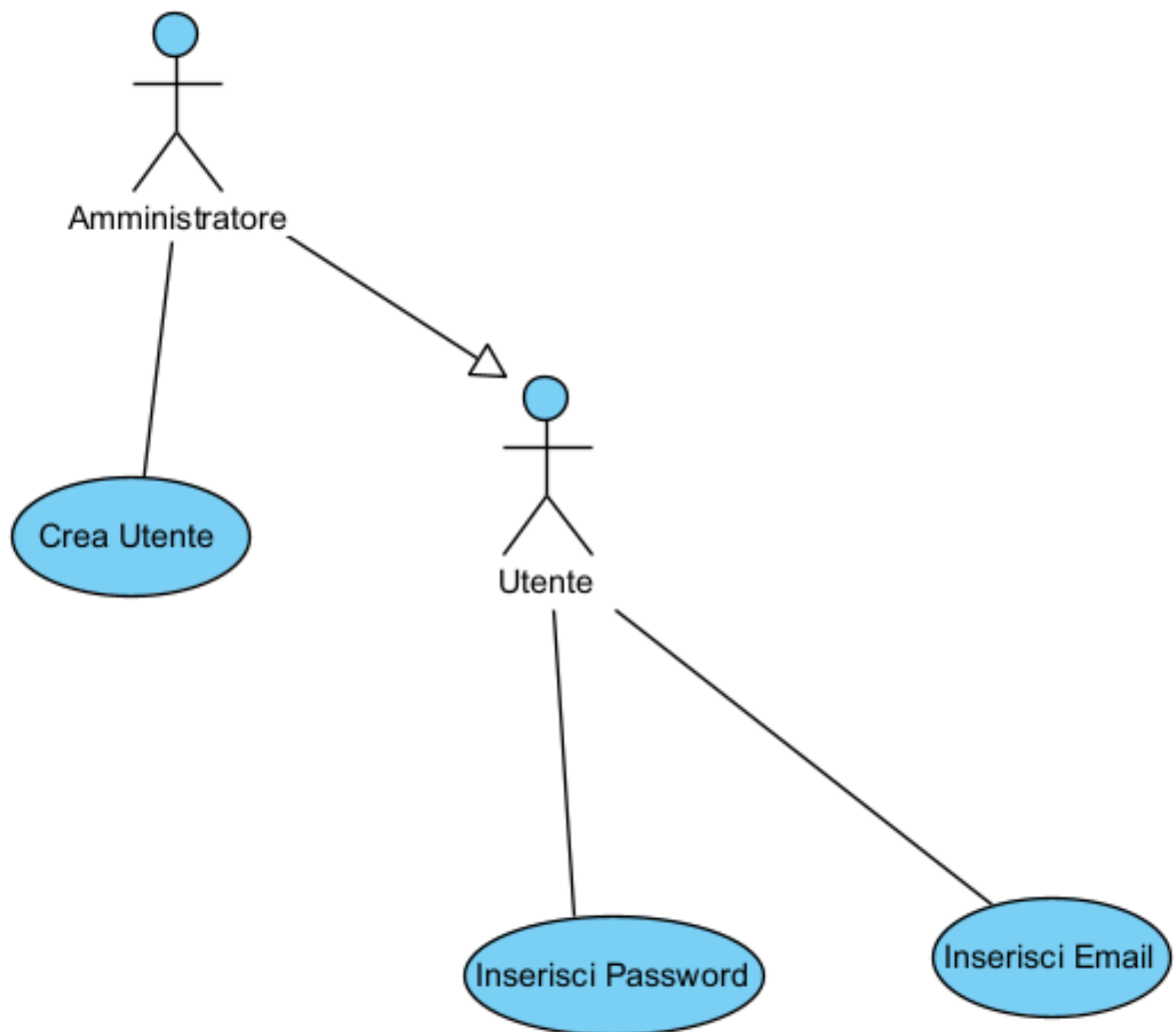


Figura 1: Use Case Diagram - Autenticazione Utente

3.2 Segnalare Issue (Requisito 2)

Tutti gli utenti autenticati possono segnalare una issue indicando almeno un titolo e una descrizione. Alcuni utenti potrebbero voler specificare anche una priorità e sarebbe gradita

la possibilità di allegare un'immagine. Le issue possono essere di diverso tipo: question (per richieste di chiarimenti), bug (per segnalare malfunzionamenti), documentation (per segnalare problemi relativi alla documentazione), e feature (per indicare la richiesta o il suggerimento di nuove funzionalità). Le issue create sono inizialmente nello stato “todo”.

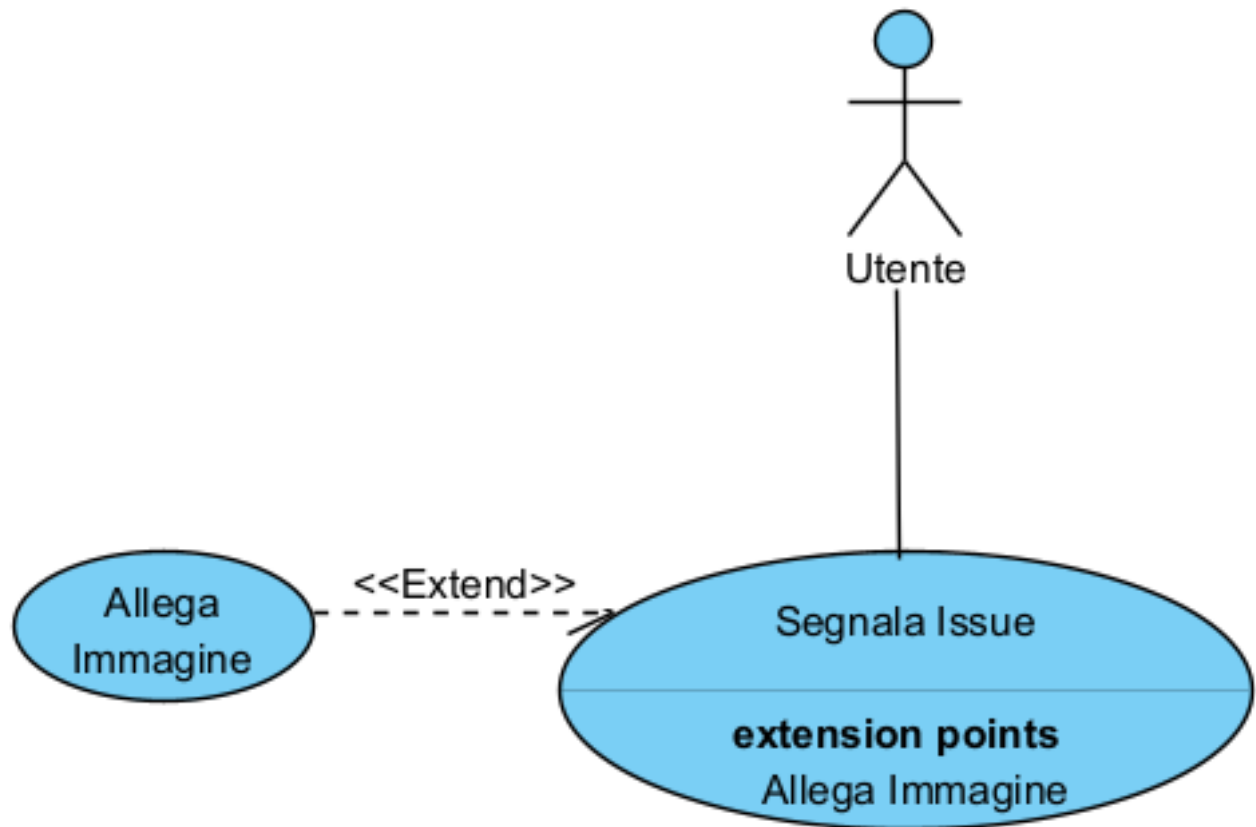


Figura 2: Use Case Diagram - UC2 Segnalare Issue

3.3 Visualizzare Vista Riepilogativa Issue (Requisito 3)

Il sistema deve offrire una vista riepilogativa delle issue, con la possibilità di filtrare o ordinare i risultati in base a criteri come tipologia, stato, priorità o altri parametri rilevanti.

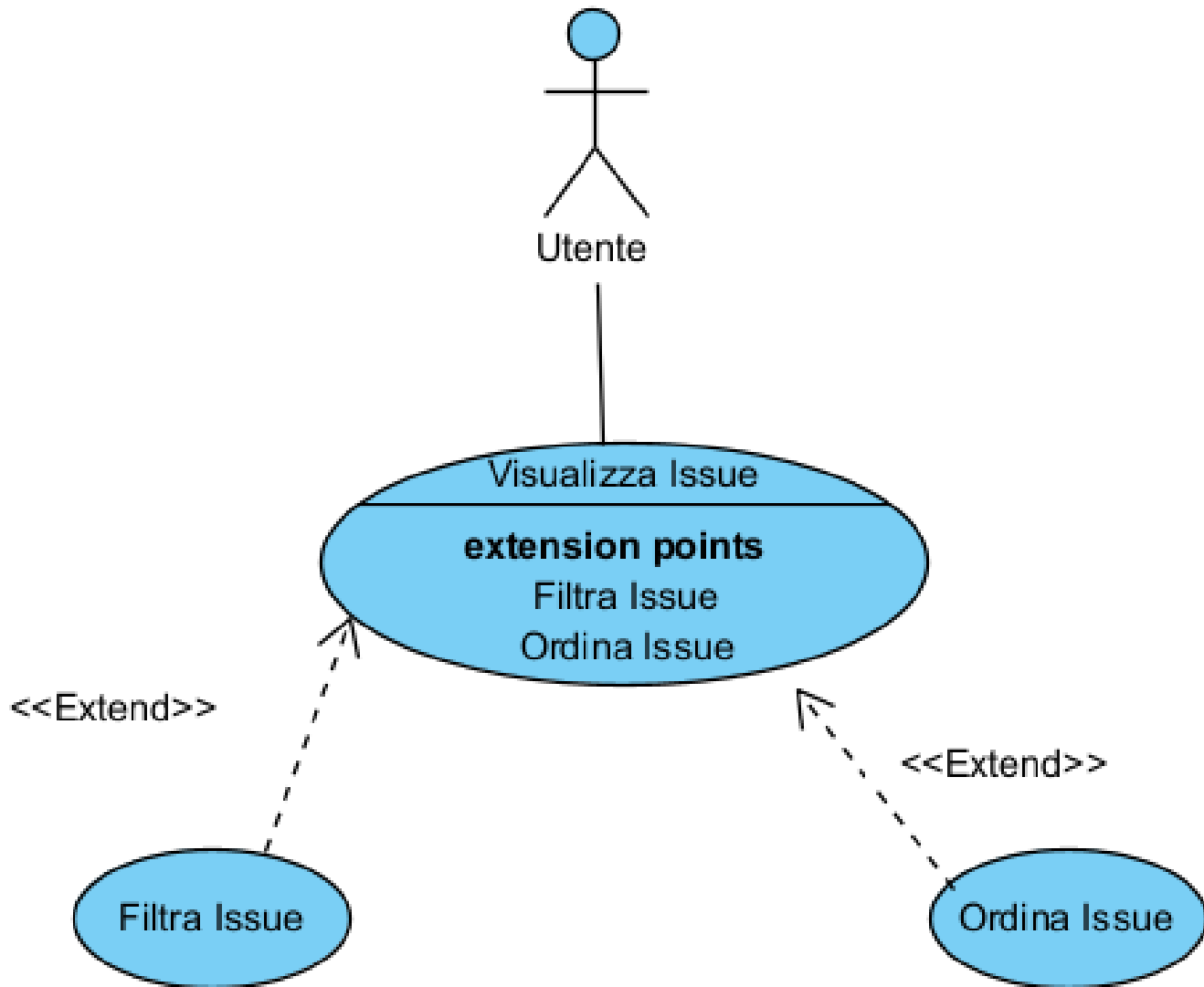


Figura 3: Use Case Diagram - UC3 Visualizzare Vista Riepilogativa Issue

3.4 Assegnare Bug (Requisito 4)

Ogni bug dovrebbe poter essere assegnato (da un amministratore) a un membro del team. Quando viene assegnato un bug a un utente, quest'ultimo riceve una notifica. Gli utenti possono visualizzare i bug loro assegnati.

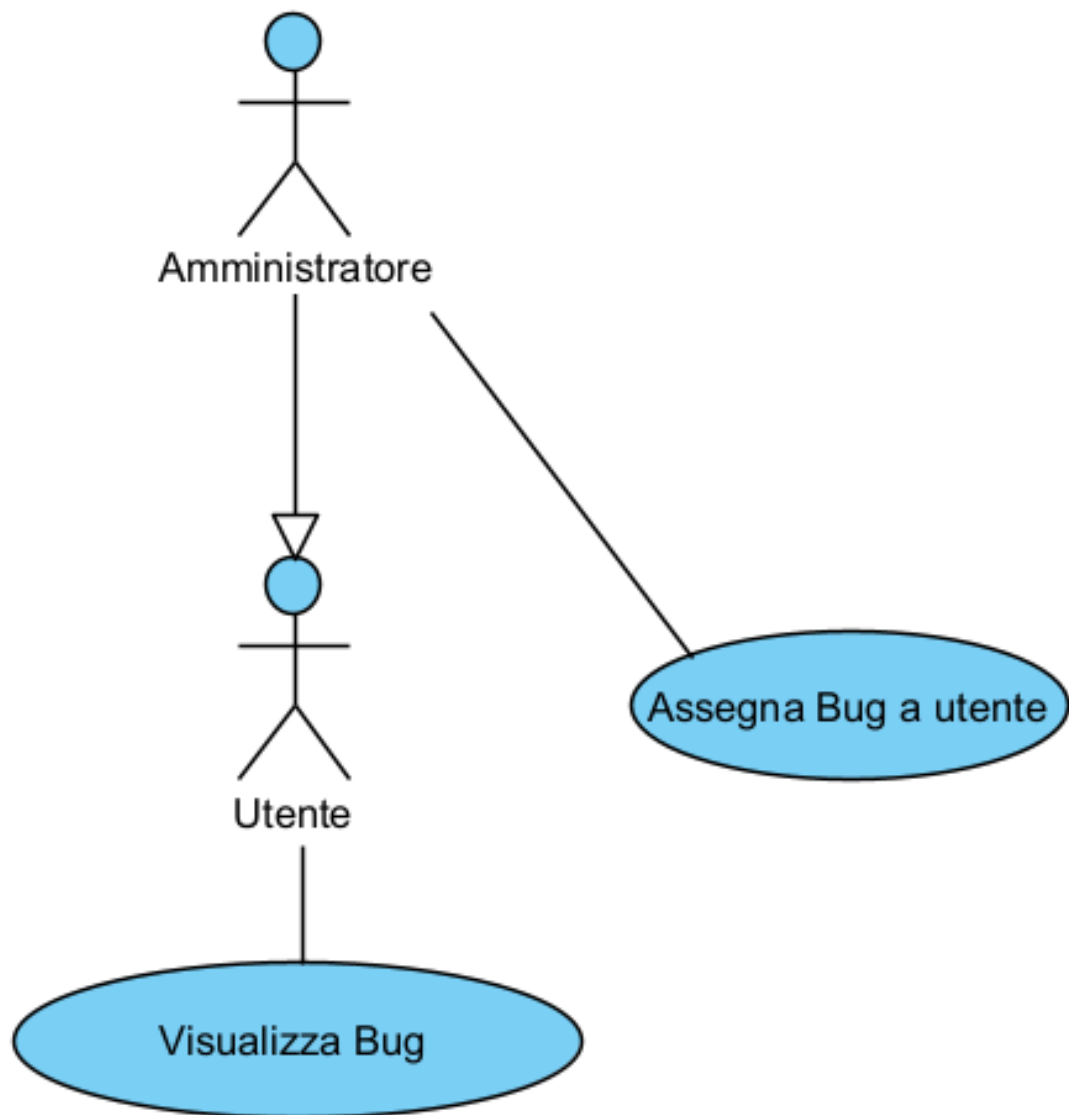


Figura 4: Use Case Diagram - UC4 Assegnare Bug

3.5 Modificare Bug Assegnato (Requisito 9)

Gli utenti devono poter modificare solo i bug a loro assegnati, mentre gli amministratori devono avere accesso completo.

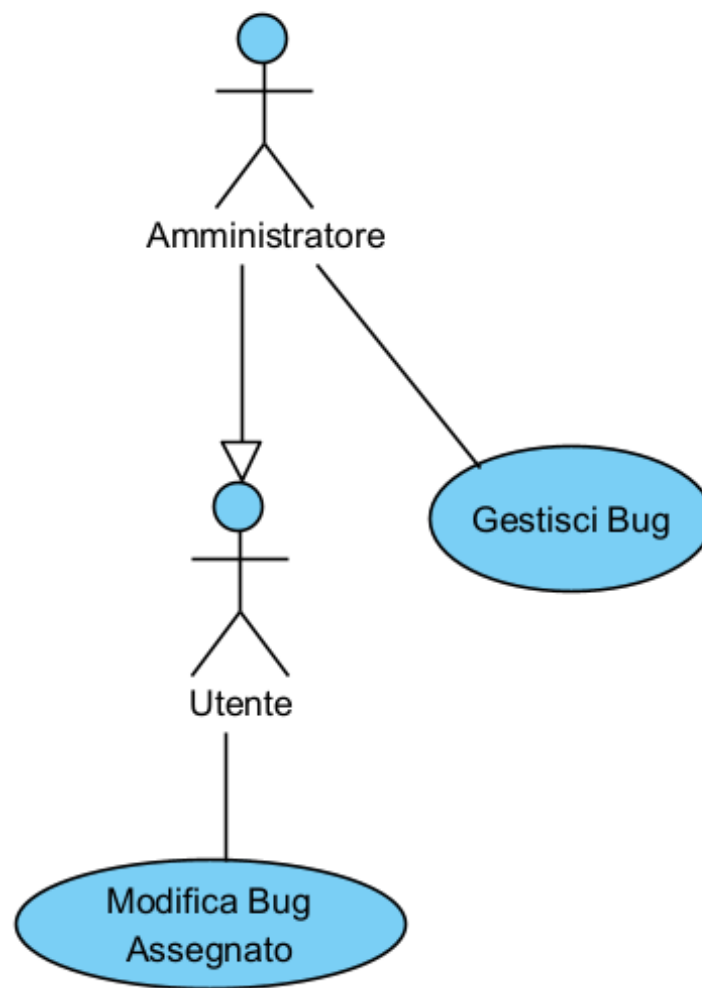


Figura 5: Use Case Diagram - UC9 Modificare Bug Assegnato

3.6 Impostare Scadenza Issue (Requisito 18)

Gli amministratori possono impostare scadenze opzionali per la risoluzione di una certa issue.

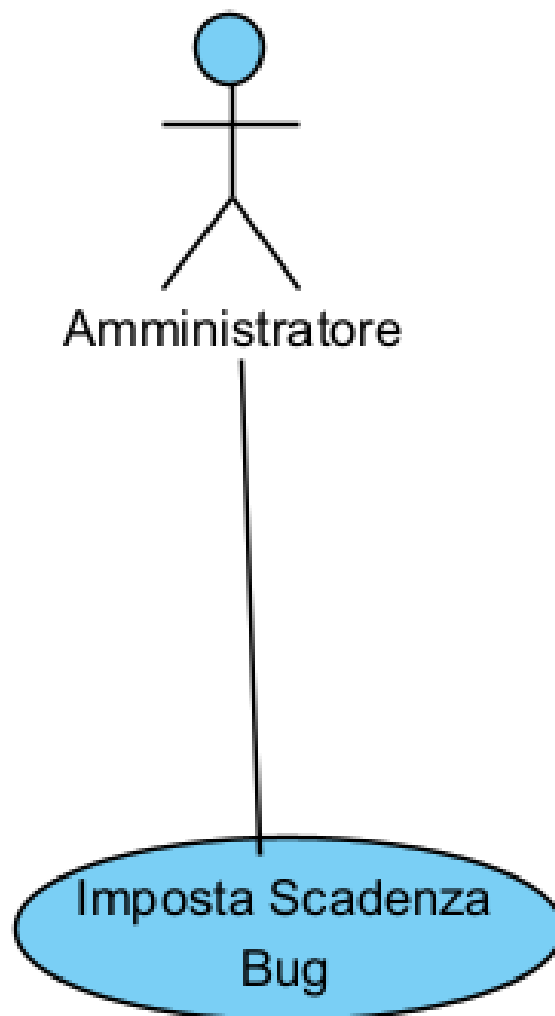


Figura 6: Use Case Diagram - UC18 Impostare Scadenza Issue

4 Individuazione e Caratterizzazione degli Utenti (Attori) del Sistema

1. Utente Autenticato

Descrizione

Rappresenta qualsiasi membro del team di sviluppo che ha completato con successo il processo di autenticazione nel sistema BugBoard26. Costituisce la categoria base di utilizzatori del sistema con permessi standard.

Caratteristiche

- Possiede credenziali valide (email e password) registrate nel sistema
- Ha superato il processo di autenticazione
- Appartiene al team di lavoro

Responsabilità e Capacità

- **Segnalazione bug:** Può creare nuove segnalazioni di problemi, specificando titolo, descrizione, tipologia (enhancement, bug, documentation, feature) e priorità
- **Visualizzazione:** Può accedere alla vista riepilogativa di tutti i bug del progetto
- **Filtro e ricerca:** Può applicare filtri per tipologia, stato, priorità e altri parametri rilevanti
- **Ordinamento:** Può ordinare i risultati secondo criteri specifici
- **Modifica limitata:** Può modificare esclusivamente i bug a lui assegnati (titolo, descrizione, tipologia, priorità, stato)
- **Gestione scadenze:** Può impostare scadenze opzionali per la risoluzione di bug
- **Notifiche:** Riceve notifiche automatiche quando viene assegnato un bug

Vincoli

- Non può modificare bug assegnati ad altri membri del team
- Non può gestire gli utenti del sistema
- Non può assegnare bug ad altri utenti
- Richiede autenticazione per accedere a qualsiasi funzionalità

2. Amministratore

Descrizione

Rappresenta un utente con privilegi elevati che ha responsabilità di gestione e supervisione dell'intero sistema BugBoard26. È una specializzazione dell'Utente Autenticato, ereditandone tutte le capacità e aggiungendo funzionalità amministrative.

Caratteristiche

- Possiede tutte le caratteristiche di un Utente Autenticato
- Ha un ruolo specifico di "Amministrazione" assegnato nel sistema
- Al momento dell'installazione del sistema esiste già un account amministratore con credenziali predefinite

Responsabilità e Capacità

Capacità Ereditate (da Utente Autenticato)

- Tutte le funzionalità disponibili agli utenti standard

Capacità Amministrative Esclusive

- **Gestione utenti:**
 - Creare nuovi account utente
 - Specificare email, password e ruolo per ogni utente
 - Assegnare ruoli (Utente standard o Amministratore)
- **Modifica completa:**
 - Modificare qualsiasi bug del sistema indipendentemente dall'assegnatario
 - Accesso completo a tutte le segnalazioni
- **Assegnazione bug:**
 - Assegnare bug a qualsiasi membro del team
 - Riassegnare bug già assegnati
 - Il sistema notifica automaticamente l'utente quando riceve un'assegnazione

Vincoli

- Richiede autenticazione come per gli utenti standard

Relazione con altri Attori

- **Generalizzazione:** L'Amministratore è una specializzazione dell'Utente Autenticato
- **Eredità:** Eredita tutte le capacità dell'Utente Autenticato più le funzionalità amministrative

3. Sistema (Attore Secondario)

Descrizione

Rappresenta il sistema BugBoard26 stesso quando agisce autonomamente per eseguire operazioni automatiche, in particolare per la gestione delle notifiche.

Caratteristiche

- Attore non umano
- Opera in modo automatico senza intervento diretto degli utenti
- Reagisce a eventi specifici del sistema

Responsabilità e Capacità

- **Invio notifiche automatiche:** Quando un amministratore assegna un bug a un utente, il sistema invia automaticamente una notifica all'utente assegnatario
- **Gestione eventi:** Rileva gli eventi di assegnazione e attiva le corrispondenti notifiche

Interazioni

- Collabora con l'Amministratore durante il processo di assegnazione bug
- Interagisce con gli Utenti Autenticati per recapitare le notifiche

5 Requisiti Non-Funzionali e di Sistema

I requisiti non-funzionali definiscono le proprietà qualitative del sistema BugBoard26. A differenza dei requisiti funzionali che descrivono cosa il sistema deve fare, questi definiscono come il sistema deve comportarsi in termini di performance, sicurezza, affidabilità, usabilità e vincoli di progetto.

5.1 Sicurezza

5.1.1 1. Autenticazione e Crittografia

Le credenziali degli utenti devono essere trasmesse e memorizzate in modo sicuro. Le password devono essere crittografate nel database.

5.1.2 2. Controllo Accessi

Il sistema deve verificare i permessi prima di ogni operazione critica. Gli utenti normali non devono poter modificare bug altrui o accedere a funzionalità amministrative. Questo controllo deve avvenire sia nel client che nel server.

5.2 Performance

5.2.1 3. Tempi di Risposta

L'applicazione deve essere responsiva. Le operazioni comuni (caricare la lista dei bug, filtrare, cercare) devono completarsi in massimo 2-3 secondi. Le operazioni di salvataggio (creare una nuova issue, modificare un bug) devono essere completate entro 3 secondi. Questo garantisce un'esperienza utente fluida senza frustrazioni dovute a caricamenti lenti.

5.2.2 4. Caricamento e Filtraggio Veloce

La visualizzazione della lista dei bug e l'applicazione di filtri devono essere rapidi e fluidi. Anche con centinaia/migliaia di issue nel sistema, il caricamento della pagina non deve superare 2 secondi e i filtri devono rispondere immediatamente. L'applicazione deve implementare paginazione per gestire efficientemente grandi quantità di dati.

5.3 Affidabilità e Disponibilità

5.3.1 5. Gestione Errori

Quando accadono errori (server non raggiungibile, ecc.), l'applicazione deve gestirli elegantemente mostrando messaggi chiari all'utente. Deve essere sempre possibile tornare a uno stato stabile.

5.4 Usabilità

5.4.1 6. Interfaccia Intuitiva

L'interfaccia deve essere facile da usare. Le operazioni più comuni devono essere raggiungibili rapidamente. I messaggi di errore devono spiegare chiaramente cosa è andato storto

e come risolverlo. La terminologia usata deve essere coerente e corrispondere al glossario del progetto.

5.4.2 7. Design Responsive

Il design deve adattarsi automaticamente a diverse dimensioni di schermo (desktop, tablet, mobile).

5.5 Architettura

5.5.1 8. Separazione Front-end e Back-end

Il front-end e il back-end devono essere completamente separati. La comunicazione deve avvenire esclusivamente tramite API (HTTP REST). Il back-end deve essere l'unica fonte di verità per lo stato del sistema e la persistenza dei dati.

5.5.2 9. Containerizzazione e Deployment

Il back-end deve essere distribuito in un container Docker. Questo garantisce che l'applicazione funzioni allo stesso modo in sviluppo, test e produzione. Il Dockerfile deve essere incluso nel repository insieme al codice sorgente. L'applicazione deve essere deployabile su servizi cloud (es. AWS, Google Cloud o Azure).

5.6 Vincoli di Implementazione e Sviluppo

5.6.1 10. Linguaggi e Paradigmi

L'implementazione del sistema deve avvenire obbligatoriamente utilizzando linguaggi di programmazione Object-Oriented, sia per il back-end che per il front-end (ove applicabile).

5.6.2 11. Testing e Qualità del Codice

Le suite di test automatizzate devono essere sviluppate mediante framework della famiglia xUnit. Il codice sorgente del back-end deve essere sottoposto ad analisi statica tramite strumenti di qualità (es. SonarQube) per la generazione di metriche e report.

5.6.3 12. Indipendenza della Logica (Divieto MBaaS esclusivo)

La logica applicativa e la persistenza dei dati non possono essere gestite esclusivamente tramite servizi esterni (es. Firebase, Supabase). Il sistema deve implementare una logica di business proprietaria nel back-end supportata da un'adeguata progettazione Object-Oriented.

5.6.4 13. Strumenti di Progettazione

La fase di analisi e progettazione deve essere supportata dall'utilizzo di strumenti CASE (es. Visual Paradigm) per la produzione dei diagrammi UML.

6 Formalizzazione Caso d'Uso Significativo

USE CASE #9 - Modificare Bug

Informazioni Generali

USE CASE #9	Modificare Bug
Goal in Context	Un utente autenticato (Utente o Amministratore) vuole modificare le informazioni di un bug esistente nel sistema per aggiornarne lo stato, correggere informazioni o aggiungere dettagli.
Preconditions	• L'utente ha effettuato l'autenticazione nel sistema • Esiste almeno un bug nel sistema • Per l'Utente: il bug deve essere assegnato a lui • Per l'Amministratore: può modificare qualsiasi bug
Success End Condition	Il bug è stato modificato con successo e le modifiche sono salvate nel sistema.
Failed End Condition	Il bug non viene modificato e rimane nello stato precedente. Il sistema può mostrare un messaggio di errore.
Primary Actor	Utente Autenticato (può essere Utente o Amministratore)
Trigger	L'utente seleziona un bug dalla lista e sceglie l'opzione di modifica

Main Scenario

Step	Utente	System
1	Accede alla lista dei bug e seleziona un bug da modificare	
2		Verifica che l'utente abbia i permessi per modificare il bug selezionato
3		Mostra la schermata di modifica del bug con i campi editabili (Titolo, Descrizione, Tipologia, Priorità, Stato) precompilati con i valori attuali
4	Modifica uno o più campi del bug (es. cambia lo stato da "In Progress" a "Done", aggiorna la descrizione, modifica la priorità)	
5	Conferma le modifiche cliccando sul pulsante "Salva"	
6		Valida i dati inseriti (titolo non vuoto, campi nel formato corretto)
7		Salva le modifiche nel database
8		Aggiorna la cronologia delle modifiche registrando i campi modificati

Step	Utente	System
9		Mostra un messaggio di conferma "Bug modificato con successo"
10		Reindirizza l'utente alla vista dei bug

Extensions

Extension #1 - Dati non validi

Step	Utente	System
6a	<L'utente ha lasciato il campo titolo vuoto o ha inserito dati non validi>	
		Rileva che i dati non sono validi
		Mostra messaggi di errore specifici accanto ai campi problematici (es. "Il titolo è obbligatorio", "Seleziona una tipologia valida")
		Mantiene la schermata di modifica aperta con i dati inseriti
		<i>Ritorna allo step 4 del Main Scenario</i>

Extension #2 - Annullamento modifica

Step	Utente	System
4a	Decide di non salvare le modifiche e clicca su "Annulla" o "Indietro"	
		Scarta tutte le modifiche non salvate
		Reindirizza l'utente alla vista precedente (lista bug) senza salvare alcuna modifica
		<i>Il caso d'uso termina</i>

Extension #3 - Errore di sistema durante il salvataggio

Step	Utente	System
7a	Si verifica un errore di connessione al database o altro errore tecnico	
		Rileva l'errore durante il tentativo di salvataggio
		Esegue il rollback della transazione per mantenere la consistenza dei dati
		Mostra un messaggio di errore "Impossibile salvare le modifiche. Riprova più tardi."
		Mantiene la schermata di modifica aperta con i dati inseriti dall'utente
		<i>Ritorna allo step 4 del Main Scenario, permettendo all'utente di riprovare</i>

Extension #4 - Amministratore modifica bug assegnato ad altri

Step	Amministratore	Utente Assegnatario	System
8a	L'amministratore modifica un bug assegnato ad un altro utente		
			Registra la modifica nella cronologia con l'ID dell'amministratore
			Invia una notifica all'utente assegnatario informandolo della modifica effettuata dall'amministratore
		Riceve la notifica	
			<i>Continua con lo step 9 del Main Scenario</i>

Mockup requisito 9

WIREFRAME

BUGBOARD26

Modifica Bug

Titolo

Errore nel calcolo della priorità

Descrizione

Il campo priorità mostra valori errati dopo l'aggiornamento manuale.

Tipologia

bug

Priorità

Alta

Stato

in progress

Attuale: in progress

Salva modifiche

Annulla

Figura 7: Wireframe del requisito

Appendice: Informazioni Progetto

Stack Tecnologico

- **Front-end:** Angular
- **Back-end:** Spring Boot
- **Database:** PostgreSQL
- **Containerizzazione:** Docker
- **Comunicazione:** API REST JSON su HTTP

Team

- Fabrizio Padulano (N86005058)
- Federico Padulano (N85005043)

Vincoli di Implementazione

- Back-end e front-end completamente separati
- Nessun utilizzo di servizi MBaaS (Firebase, Supabase, etc.)
- HTTP Rest obbligatorio
- Docker obbligatorio per back-end
- Linguaggio Object-Oriented

7 Documento di Design del sistema

7.1 Analisi dell'architettura

Il software sviluppato si presenta come un'architettura **Client-Server distribuita**. Tale decisione è stata adottata per garantire la separazione netta tra la logica di presentazione (Front-end) e la gestione dei dati e della business logic (Back-end).

I componenti comunicano attraverso **API REST** su protocollo HTTP. Per garantire la portabilità e la facilità di deployment, il componente Back-end è stato containerizzato utilizzando **Docker**.

7.1.1 Client (Front-end)

Il client è stato sviluppato come *Single Page Application* (SPA) utilizzando il framework **Angular**. La scelta di Angular deriva dalla sua robustezza nella creazione di interfacce web modulari e dalla sua architettura *Component-Based*.

L'architettura si basa su una gerarchia di componenti e servizi:

- **Componenti (Components):** Gestiscono la vista (HTML/CSS) e la logica di interazione con l'utente. Per ogni componente si è cercato di renderlo isolato e riutilizzabile.

- **Servizi (Services):** Classi che contengono la logica di recupero dati e comunicazione con il Server. Grazie alla *Dependency Injection* nativa di Angular, i servizi vengono iniettati nei componenti, mantenendo il codice separato.
- **Modelli (Interfaces/Models):** Definiscono la struttura dei dati (es. l'interfaccia Bug o User) garantendo il controllo dei tipi grazie a TypeScript.

7.1.2 Server (Back-end)

Per la creazione del back-end è stato utilizzato Java con il framework **Spring Boot**. L'architettura del Server segue il pattern **Layered** (a strati), suddivisa in:

Controller Layer: (es. `BugController`) Gestisce l'interfaccia REST. Riceve le richieste HTTP dal Client, le valida e delega l'elaborazione al livello sottostante.

Service Layer: (es. `BugService`) Contiene la vera *Business Logic*. Qui avvengono i controlli di sicurezza, i calcoli e l'orchestrazione delle operazioni.

Repository Layer: (es. `BugRepository`) Gestisce l'interazione con il database PostgreSQL tramite Spring Data JPA. Le query SQL vengono astratte tramite metodi Java.

Entity Layer: Classi Java che mappano direttamente le tabelle del database (ORM - *Object Relational Mapping*).

Architettura di Deployment e Containerizzazione

Per massimizzare la portabilità e garantire la consistenza tra gli ambienti di sviluppo e produzione, l'intero stack tecnologico è stato containerizzato utilizzando **Docker**.

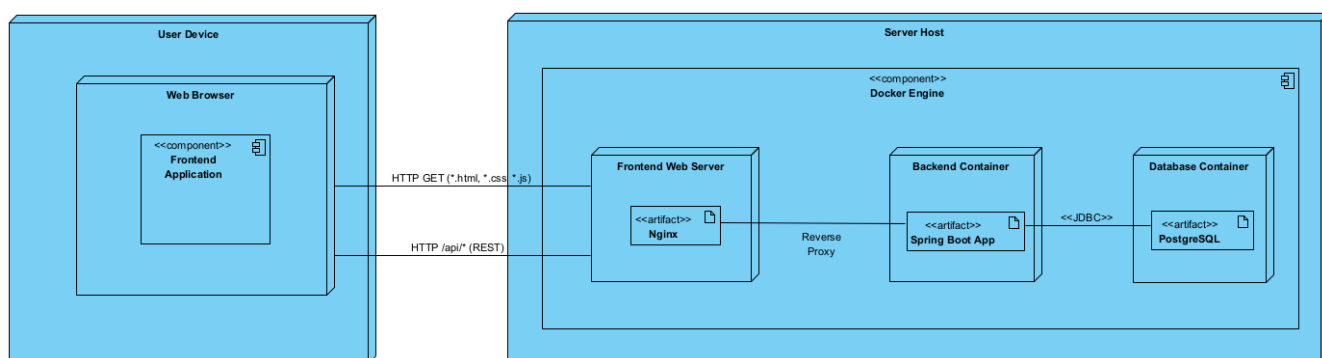


Figura 8: Diagramma dell'Architettura Full-Stack e Flusso dei Dati

L'architettura prevede tre container orchestrati tramite **Docker Compose**:

- **Frontend Container:** Ospita un web server (Nginx) configurato per servire i file statici generati dalla build di produzione di Angular. Questo disaccoppia il deployment dall'ambiente di sviluppo: non è necessario avere Node.js installato sulla macchina host per eseguire il front end.

- **Backend Container:** Esegue l'applicazione Spring Boot all'interno di un ambiente Java isolato, esponendo le API RESTful necessarie al client.
- **Database Container:** Esegue un'istanza di PostgreSQL con persistenza dei dati garantita tramite volumi Docker.

Il flusso di esecuzione prevede che il browser interagisca esclusivamente con il Web Server Nginx, configurato come Reverse Proxy. Nginx agisce come unico punto di ingresso (Gateway): gestisce il routing delle richieste, servendo i file statici dell'applicazione Frontend (Angular) e inoltrando in modo trasparente le chiamate API verso il Backend Container, che risiede su una rete interna non esposta pubblicamente. Questa architettura risolve le problematiche di Cross-Origin Resource Sharing (CORS) e migliora la sicurezza isolando il servizio di backend

7.2 Architetture di Riferimento

Il sistema *BugBoard* integra diversi modelli architetturali. Di seguito vengono descritte le motivazioni tecniche che hanno guidato l'implementazione di ciascuno di essi nel progetto.

7.2.1 Architettura a 3 Livelli (3-Tier) e Strati Chiusi

Il sistema è stato strutturato fisicamente su tre livelli distinti (*Presentation*, *Logic*, *Data*) e logicamente su strati chiusi.

- **Il perché della scelta:** Questa struttura è stata adottata per garantire il massimo isolamento dei componenti. Utilizzando Nginx come Reverse Proxy nello strato di *Presentation*, abbiamo creato una "barriera" che impedisce all'utente di interfacciarsi direttamente con il backend.
- **Vantaggio tecnico:** Un utente non può invocare direttamente funzioni del database senza passare per i controlli di sicurezza e le validazioni scritte nel *Logic Tier* (Spring Boot).

7.2.2 Architettura a Repository (Repository Pattern)

L'accesso ai dati è stato isolato tramite l'implementazione di un'architettura a Repository, mediata da Spring Data JPA.

- **Il perché della scelta:** Abbiamo scelto questo pattern per disaccoppiare la logica di business (i Service) dalla tecnologia di persistenza (SQL/PostgreSQL). I Service non sanno "come" i dati vengono salvati, ma sanno solo a quale Repository chiederli.

7.3 Scelte tecnologiche adottate

7.3.1 Core Technologies

- **Spring Boot (Back-end):** Scelto per l'ecosistema e la gestione automatica delle dipendenze. Riduce drasticamente il codice *boilerplate* e facilita la creazione del backend.

- **Angular (Front-end):** Framework completo che fornisce strumenti nativi per il routing, le chiamate HTTP e la gestione dei form, dependency injection e tante altre funzionalità.
- **PostgreSQL (Database):** La scelta tecnologica è ricaduta su questo DBMS poiché è stato il principale oggetto di studio durante i corsi di Basi di Dati e Object Orientation.

7.3.2 Infrastruttura e Deployment

- **Docker:** Il back-end è stato pacchettizzato in un Container Docker per garantire che il software giri allo stesso modo in ambiente di sviluppo, test e produzione.
- **GitHub:** Utilizzato per il versionamento del codice, permettendo al team di lavorare in parallelo e tracciare ogni modifica.
- **SonarQube:** Utilizzato per l'analisi statica del codice del back-end al fine di identificare bug, vulnerabilità e debito tecnico.

8 Documento di Design del Software, documentazione del processo di sviluppo, e artefatti software.

8.1 Schema per la persistenza dati

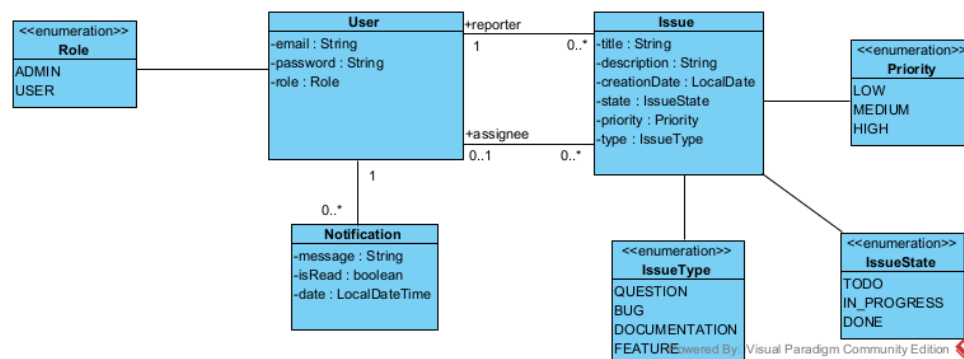


Figura 9: Diagramma UML delle Entità (Domain Model)

Il modello di dominio, rappresentato nel diagramma UML soprastante, si articola nelle seguenti componenti:

- **Entità User:** Rappresenta gli utenti del sistema. Include attributi per l'autenticazione (**email**, **password**) e un attributo **role** basato sull'enumerazione **Role** (ADMIN, USER) per la gestione del controllo degli accessi.
- **Entità Issue:** Rappresenta la segnalazione di un bug o di una funzionalità. È l'entità centrale e memorizza lo stato (**IssueState**), la priorità (**Priority**) e la tipologia (**IssueType**) tramite tipi enumerati per garantire la consistenza dei dati.
- **Entità Notification:** Gestisce le comunicazioni automatiche verso gli utenti, memorizzando il messaggio, il timestamp di invio e lo stato di lettura (**isRead**).

Relazioni e Molteplicità

Le associazioni tra le classi riflettono i vincoli di integrità referenziale del database:

- **Issue – User (Reporter):** Associazione multi-a-uno (**ManyToOne**) che identifica l'utente che ha creato la segnalazione.
- **Issue – User (Assignee):** Associazione multi-a-uno (**ManyToOne**) opzionale, che lega la *issue* al membro del team incaricato della risoluzione.
- **Notification – User (Recipient):** Relazione multi-a-uno (**ManyToOne**) che associa ogni notifica al suo destinatario univoco.

8.2 Diagramma delle classi di design

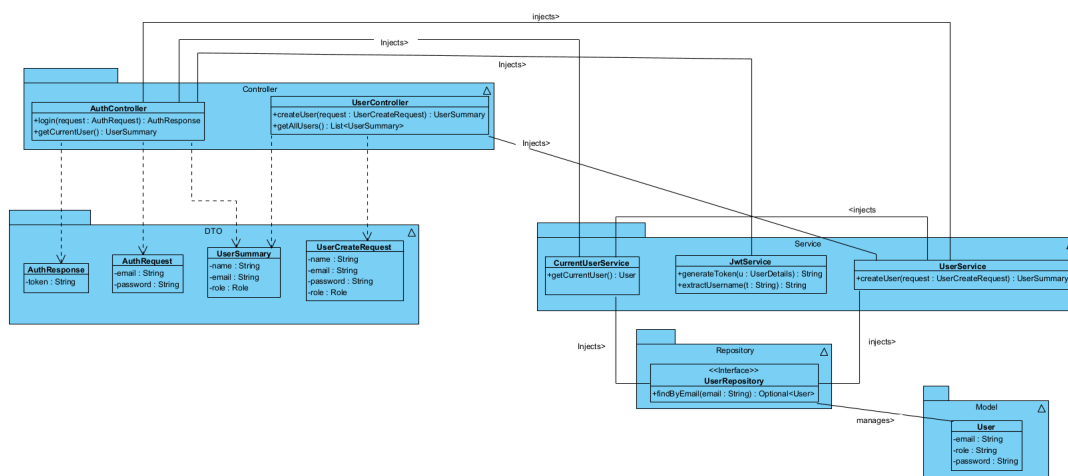


Figura 10: Diagramma delle classi: Autenticazione e Sicurezza

Il diagramma in Figura 10 illustra l'architettura del modulo responsabile dell'identificazione degli attori e della sicurezza. La struttura segue rigorosamente una **Layered Architecture** (Architettura a livelli), dove il flusso delle dipendenze è chiaramente definito per garantire manutenibilità e protezione dei dati.

Analizzando i componenti e le loro interazioni, emergono tre aspetti fondamentali:

- **Orchestrazione dell'Autenticazione (AuthController):** Il controller di autenticazione non si limita a gestire il login, ma funge da coordinatore tra diversi servizi critici. Come mostrato dalle relazioni di associazione, esso inietta:
 - **UserDetailsService:** per il caricamento delle credenziali di sicurezza.
 - **JwtService:** per la generazione del token stateless.
 - **CurrentUserService:** per fornire al frontend i dettagli dell'utente attualmente autenticato tramite il DTO **UserSummary**.
- **Sicurezza nella Business Logic (UserService):** Una caratteristica di questo design è il collegamento tra **UserService** e **CurrentUserService**. Il servizio di gestione utenti non si occupa solo della persistenza tramite il **UserRepository**, ma interroga il contesto di sicurezza prima di eseguire operazioni sensibili. Ad esempio,

il metodo di creazione di un nuovo utente verifica l'identità dell'operatore loggato per garantire che solo un utente con ruolo ADMIN possa procedere, spostando i controlli di autorizzazione direttamente nel cuore della logica di business.

- **Disaccoppiamento tramite DTO:** Le dipendenze tratteggiate verso le classi `AuthRequest`, `AuthResponse`, `UserCreateRequest` e `UserSummary` evidenziano l'uso sistematico dei **Data Transfer Objects**. Questo strato impedisce all'entità persistente `User` di essere esposta direttamente nelle API, proteggendo campi sensibili (come la password hashata) e permettendo all'interfaccia pubblica di evolvere indipendentemente dal database.

Design Pattern Applicati in questo modulo

Dependency Injection: Il sistema sfrutta l'Inversione del Controllo (IoC). I ruoli architetturali sono così ripartiti:

- **Client** (`AuthController`): La classe che necessita di funzionalità esterne per operare.
- **Service** (`UserDetailsService` / `JwtService`): Le dipendenze che vengono iniettate e forniscono la logica specifica.
- **Injector** (**Spring Container**): Il framework che gestisce il ciclo di vita e inietta le istanze tramite il costruttore (generato da `@RequiredArgsConstructor`).

Builder Pattern: L'istanziamento di oggetti complessi avviene separando la costruzione dalla rappresentazione. La mappatura sui componenti del progetto è la seguente:

- **Product** (`User` / `UserSummary`): L'oggetto finale complesso che deve essere creato.
- **ConcreteBuilder** (`UserBuilder`): La classe interna statica (generata da Lombok) che accumula i parametri passo dopo passo.
- **Director** (`UserService`): Il servizio che orchestra la costruzione, definendo quali campi impostare (es. criptando la password) prima di invocare il metodo `build()`.

Singleton Pattern: Tutte le classi come `Controller`, `Service` e `Repository` sono gestite come *Singleton* dal framework. Questo garantisce che esista una sola istanza di `JwtService` o `UserRepository` in memoria, centralizzando la gestione dello stato e ottimizzando l'uso delle risorse.

8.2.1 Modulo Gestione Issue (Core Business)

La Figura 11 illustra il cuore funzionale del sistema *BugBoard26*. In questo modulo risiede la logica di business principale, dedicata alla creazione, gestione e tracciamento delle segnalazioni (Issue).

Analizzando i componenti, si evidenziano le seguenti responsabilità:

- **Gestione delle operazioni principali** (`IssueService`): Questo servizio funziona come il "cervello" o il punto di controllo centrale. Per svolgere il suo lavoro, utilizza e coordina altri componenti (tramite l'Iniezione delle dipendenze):

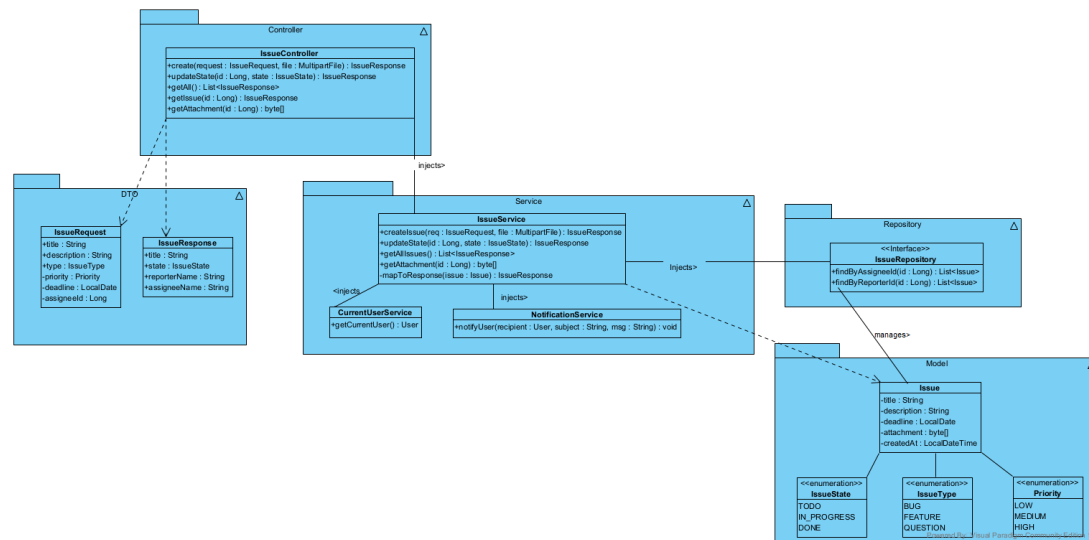


Figura 11: Diagramma delle classi: Core Business e Gestione Segnalazioni

- **IssueRepository**: serve per salvare le segnalazioni nel database.
 - **CurrentUserService**: serve per riconoscere automaticamente chi sta creando la segnalazione (il *Reporter*), assicurandosi che questa informazione sia sicura e non possa essere manipolata da chi usa l'applicazione.
 - **NotificationService**: serve per inviare avvisi (come ad esempio una notifica oppure un'email) ogni volta che lo stato di una segnalazione cambia.
- **Integrità tramite Enumerazioni**: Lo stato e la tipologia delle segnalazioni non sono stringhe libere, ma tipi fortemente tipizzati (**IssueState**, **Priority**, **IssueType**). Questo vincola i valori ammissibili a livello di compilazione, prevenendo stati inconsistenti.

Design Pattern Applicati in questo modulo

Builder Pattern (Pattern Creazionale): Data la complessità dell'entità **Issue** (che possiede molti campi opzionali come scadenze, assegnatari e allegati), viene utilizzato il Builder per costruirla.

- **Product (Issue)**: L'entità complessa da costruire.
- **Director (IssueService)**: Il metodo `createIssue` che coordina i passaggi (es. conversione del file in byte, assegnazione del reporter).
- **ConcreteBuilder**: L'implementazione interna generata da Lombok che assembla l'oggetto passo dopo passo.

Dependency Injection: Anche qui, il disaccoppiamento è garantito dall'IoC. Il **IssueService** non crea internamente le istanze dei repository o dei servizi di notifica, ma dichiara le proprie necessità nel costruttore, lasciando al container Spring il compito di risolvere e iniettare le dipendenze corrette.

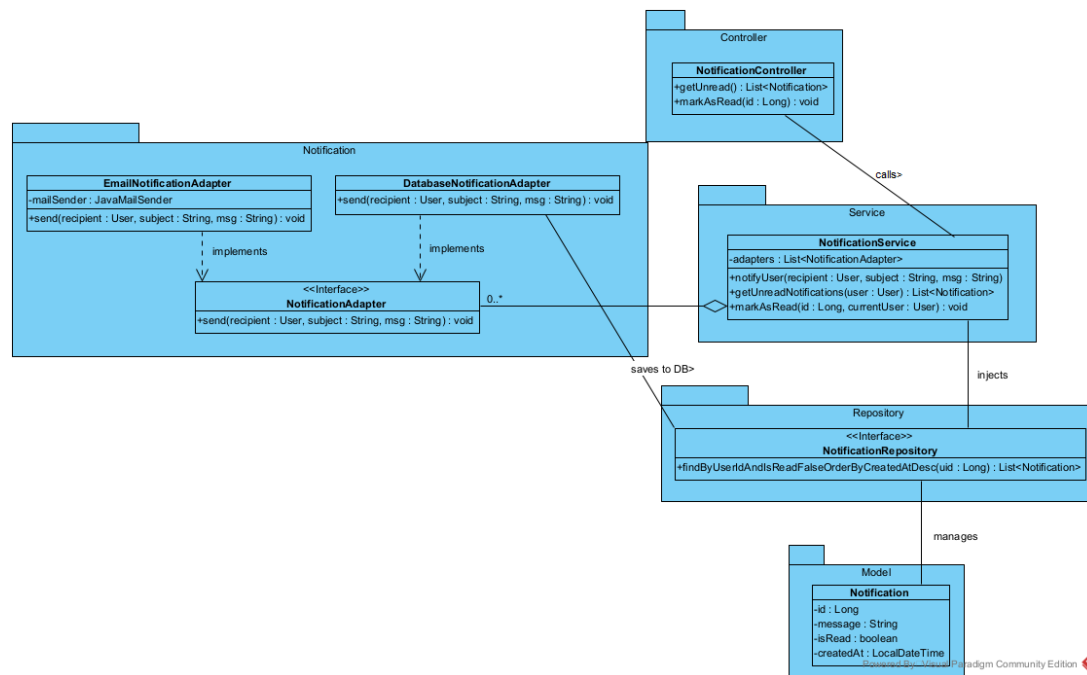


Figura 12: Diagramma delle classi: Sottosistema di Notifica

8.2.2 Modulo Notifiche

La Figura 12 illustra il sottosistema responsabile della comunicazione verso gli utenti. Questo modulo si distingue per un design altamente disaccoppiato, progettato per gestire molteplici canali di invio senza legare la logica di business a specifiche tecnologie.

Analizzando il flusso mostrato nel diagramma, si distinguono tre livelli di astrazione:

- **Il Contesto (NotificationService):** Al centro del modulo opera il servizio di notifica. A differenza di altri componenti, esso non contiene la logica “fisica” per l’invio; mantiene invece una lista di oggetti che rispettano l’interfaccia `NotificationAdapter`. Al momento dell’invio (metodo `notifyUser`), il service scorre la lista delegando l’esecuzione ai singoli adattatori.
- **L’Astrazione (Interface):** L’elemento chiave è l’interfaccia `NotificationAdapter`, che definisce il contratto generico `send()`. Il resto dell’applicazione interagisce esclusivamente con questa interfaccia, ignorando i dettagli implementativi sottostanti.
- **Le Implementazioni Concrete:** Si identificano le classi operative che estendono l’interfaccia:
 - `EmailNotificationAdapter`: gestisce l’invio tramite protocollo SMTP.
 - `DatabaseNotificationAdapter`: interagisce con il `NotificationRepository` per la persistenza delle notifiche in-app.

Analisi dei Design Pattern Applicati

Adapter Pattern: È il cuore del modulo. In questo contesto, `EmailNotificationAdapter` e `DatabaseNotificationAdapter` agiscono come **Adapters**. Essi traducono (adattano) l’interfaccia generica richiesta dal sistema (**Target**: `NotificationAdapter`)

nelle chiamate specifiche necessarie per i servizi reali (**Adaptee**: `JavaMailSender` o `NotificationRepository`). In questo schema, il `NotificationService` agisce come **Client**, interagendo solo con l'astrazione.

- Questo approccio rispetta il principio **Open/Closed**: il sistema è aperto all'estensione (posso aggiungere nuovi canali come SMS o Telegram semplicemente creando un nuovo Adapter) ma chiuso alla modifica (non dovrò mai modificare il codice del `NotificationService` per aggiungere tali canali).

Dependency Injection: Il `NotificationService` non istanzia manualmente i suoi adattatori (non usa il comando `new`). Invece, tramite l'annotazione `@RequiredArgsConstructor` di Lombok, richiede a Spring di iniettare una `List<NotificationAdapter>`.

- Grazie all'Inversione del Controllo (**IoC**), Spring individua automaticamente all'avvio tutte le classi che implementano l'interfaccia `NotificationAdapter` e le fornisce al servizio. Questo rende il sistema estremamente flessibile: il servizio invierà notifiche su tutti i canali disponibili nel progetto in modo totalmente dinamico e trasparente.

8.3 Motivazioni delle Scelte dei Design Pattern

La selezione dei pattern utilizzati nei diversi moduli non è casuale, ma risponde a specifiche problematiche architetturali emerse durante l'analisi:

Uso del Builder Pattern (Moduli 1 e 2):

Nei moduli *Gestione Utenti* e *Gestione Issue*, la sfida principale riguardava la creazione di oggetti complessi con numerosi attributi, molti dei quali opzionali (come gli allegati nelle Issue o i campi non obbligatori nei profili utente). Il **Builder Pattern** ha permesso di rendere la creazione delle istanze esplicita, dichiarando solo gli attributi necessari in un dato momento.

Uso dell'Adapter Pattern (Modulo 3):

Nel modulo *Notifiche*, il problema principale era far dialogare tra loro strumenti diversi. Il sistema doveva comunicare sia con il server delle email (tramite `JavaMail`) sia con il database (tramite `Repository JPA`). Dato che questi strumenti hanno linguaggi differenti, è stato introdotto l'**Adapter Pattern**. Questo ha permesso di creare un'interfaccia unica (`NotificationAdapter`) che fa da "traduttore", separando la logica del programma dai dettagli tecnici. In questo modo, sarà facile aggiungere in futuro nuovi canali di comunicazione senza dover modificare il codice esistente.

8.4 Gestione del Codice e Versioning

Per coordinare il lavoro tra i membri del team e mantenere uno storico affidabile delle modifiche, abbiamo utilizzato Git come sistema di controllo versione. Il codice sorgente è ospitato su una repository pubblica di GitHub.

8.4.1 Repository GitHub

Il progetto è strutturato come un *Monorepo*, contenente sia il codice del Back-end (Spring Boot) che del Front-end (Angular) e le configurazioni Docker. È possibile consultare il codice completo al seguente indirizzo:

Link al Repository:

<https://github.com/fabpadulano-unina/BugBoard26>

backend	fix per sonar
frontend	aggiunti dockerfile
.gitignore	Initial setup: Spring Boot backend + Angular frontend + D...
docker-compose.yml	aggiunti dockerfile

Figura 13: Panoramica del repository su GitHub con la struttura delle cartelle.

8.4.2 Statistiche e Flusso di Lavoro

Il grafico dei contributi mostra la frequenza dei commit e la partecipazione attiva di entrambi i membri del team.

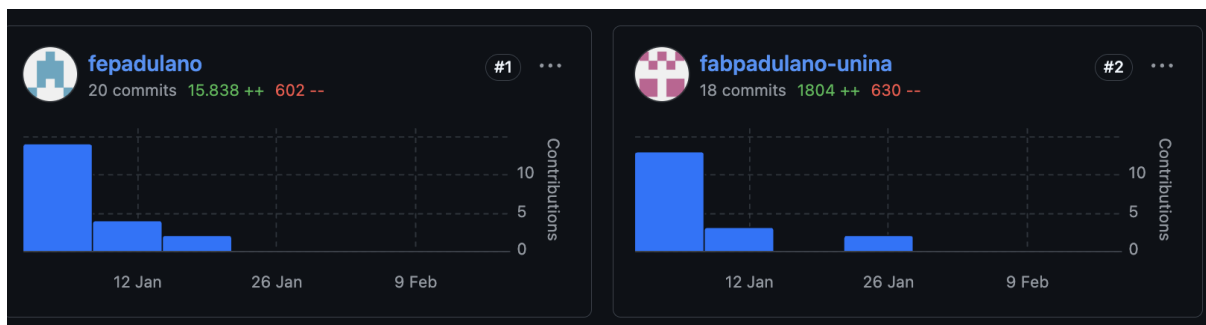


Figura 14: Grafico della frequenza dei commit e contributi del team.

Va precisato che la disparità nel volume di righe di codice aggiunte (visibile nel picco di oltre 15.000 righe per l'utente *fepadulano*) è attribuibile al commit del file di configurazione `package-lock.json`. Trattandosi di un file generato automaticamente per la gestione delle dipendenze npm, il suo elevato numero di righe non riflette la quantità di logica implementata manualmente (che è di circa 1.500 righe).

8.5 Qualità del Codice

Per garantire che il software non fosse solo funzionante ma anche ben scritto, abbiamo sottoposto l'intero codice del Back-end (Spring Boot) all'analisi statica tramite SonarQube. L'obiettivo era semplice: individuare subito eventuali bug, vulnerabilità di sicurezza o punti del codice scritti male (i cosiddetti "Code Smells") prima che diventassero un problema difficile da risolvere.

8.5.1 Analisi dei Risultati

L'analisi automatica ha confermato la solidità del codice prodotto. Come mostrato nella dashboard riepilogativa (Figura 15), il progetto ha superato il controllo qualità (*Quality Gate*), ottenendo il rating massimo (**A**) in tutte le categorie principali: Affidabilità, Sicurezza e Manutenibilità.

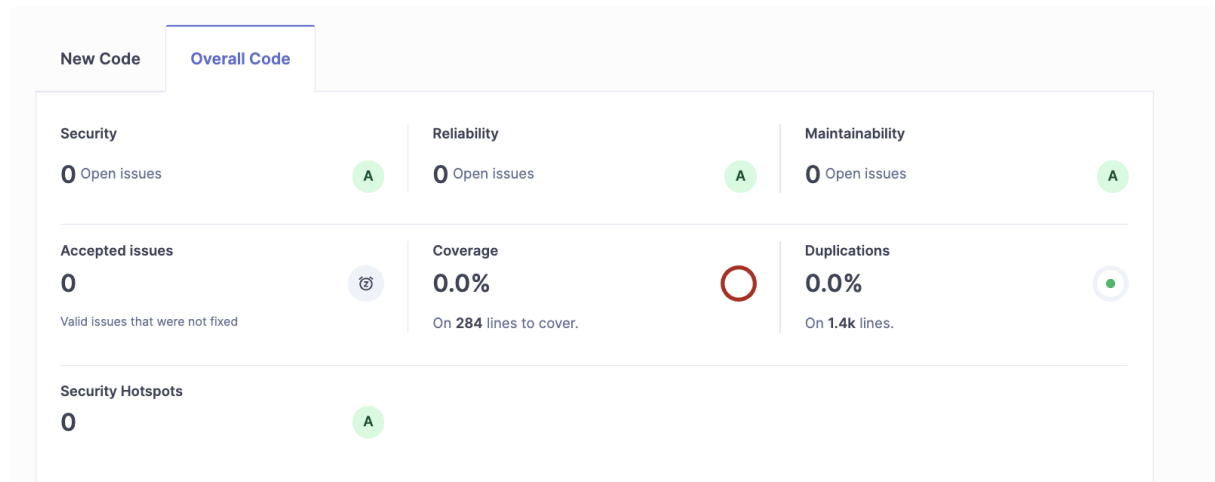


Figura 15: Panoramica di SonarQube: il codice rispetta tutti gli standard di qualità (Rating A).

Un dato particolarmente positivo è la percentuale di duplicazione del codice, che si attesta allo **0.0%**. Questo indica che abbiamo evitato la pratica del "copia-incolla", preferendo riutilizzare le classi e i metodi in modo efficiente. Inoltre, il "Debito Tecnico" (ovvero il tempo stimato per correggere le imperfezioni del codice) è risultato molto basso, garantendo che il software sarà facile da mantenere ed evolvere in futuro.

Maintainability	?	▼
Overview		
Overall Code		
Issues		0
Debt		0
Debt Ratio		0.0%
Rating		A
Effort to Reach A		0

8.6 DockerFile e file di build Automatica

In questa sezione analizziamo la configurazione utilizzata per containerizzare l'applicazione. L'obiettivo è garantire un ambiente di esecuzione identico tra sviluppo e produzione, automatizzando il processo di compilazione e deployment.

8.6.1 Backend Dockerfile

Il seguente Dockerfile definisce il processo per creare l'immagine del Backend. Viene utilizzata una strategia di **Multi-Stage Build** per ottimizzare le dimensioni dell'immagine finale (utile per ridurre i costi in cloud):

1. **Build Stage:** Utilizza un'immagine Maven completa per compilare il codice sorgente e generare il file `.jar`.
2. **Run Stage:** Copia solo l'artefatto compilato in un'immagine JRE (Java Runtime Environment) leggera, scartando i tool di compilazione non necessari in produzione.

```
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests
```

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

8.6.2 Frontend Dockerfile

Analogamente al backend, anche il frontend Angular viene gestito in due fasi. Dopo la compilazione dei file TypeScript tramite Node.js, l'applicazione statica risultante viene servita da un web server Nginx.

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build --configuration=production
```

```
FROM nginx:alpine
COPY --from=build /app/dist/frontend/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

8.6.3 Orchestrazione con Docker Compose

Il file `docker-compose.yml` definisce l'architettura completa del sistema, gestendo le reti, i volumi persistenti e le dipendenze di avvio tra i vari container.

```
services:
  # 1. Database PostgreSQL
  postgres:
    image: postgres:16
    container_name: bugboard-db
    environment:
      POSTGRES_DB: bugboard_db
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: password123
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin -d bugboard_db"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - bugboard-net

  # 2. PgAdmin
  pgadmin:
    image: dpage/pgadmin4
    container_name: pgadmin_bugboard
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@admin.com
      PGADMIN_DEFAULT_PASSWORD: root
    ports:
      - "5050:80"
    depends_on:
      - postgres
    networks:
      - bugboard-net

  # 3. Backend (Spring Boot)
  backend:
    build: ./backend
    container_name: bugboard-backend
    ports:
      - "8080:8080"
    environment:
      DB_HOST: bugboard-db
      POSTGRES_USER: admin
```

```
    POSTGRES_PASSWORD: password123
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - bugboard-net

# 4. Frontend (Angular + Nginx)
frontend:
  build: ./frontend
  container_name: bugboard-frontend
  ports:
    - "80:80"
  depends_on:
    - backend
  networks:
    - bugboard-net

volumes:
  postgres_data:
  sonar_data:

networks:
  bugboard-net:
```

Questo file configura i servizi principali del sistema:

- **Postgres:** Fornisce il database persistente. È configurato con un *Healthcheck* nativo che permette agli altri servizi di attendere che il database sia effettivamente pronto a ricevere connessioni prima di avviarsi.
- **Backend:** Il servizio Spring Boot. Utilizza la direttiva `depends_on` con `condition: service_healthy` per garantire che non si verifichino errori di connessione al database all'avvio.
- **Frontend:** Espone l'applicazione web sulla porta standard 80, comunicando con il backend attraverso la rete interna di Docker.

9 Attività di Testing

9.1 Test Plan Funzionale

Il piano di test si focalizza sulla validazione della logica di business del modulo Issue, verificando l'integrità dei dati e la corretta comunicazione tra i componenti.

ID	Funzionalità	Pre-condizioni	Risultato Atteso
TC-01	Creazione Issue con Allegato	Utente autenticato, file binario caricato correttamente.	La Issue viene salvata nel database e l'allegato è convertito in un array di byte.
TC-02	Notifica tramite Adapter	Creazione di una Issue con un assegnatario specifico.	Il sistema invia automaticamente una notifica delegando l'operazione agli adapter configurati.

Tabella 7: Casi di test funzionali per il modulo segnalazioni

9.2 Test di Unità Automatici

I test di unità sono stati realizzati utilizzando il framework **JUnit 5** e la libreria **Mockito**. L'obiettivo è isolare la logica di business dei Service simulando il comportamento delle dipendenze tramite i "Mock".

Test 1: Creazione Issue e Gestione Allegati (IssueService)

Questo test verifica che il metodo `createIssue` (che riceve due parametri non banali: `IssueRequest` e `MultipartFile`) coordini correttamente il salvataggio dei dati e l'invio della notifica all'assegnatario. In questo contesto, il Service agisce come **Director** per la costruzione dell'entità complessa.

```
@ExtendWith(MockitoExtension.class)
class IssueServiceTest {
    @Mock private IssueRepository issueRepository;
    @Mock private UserService userService;
    @Mock private CurrentUserService currentUserService;
    @Mock private NotificationService notificationService;
    @InjectMocks private IssueService issueService;

    @Test
    void createIssue_ShouldSaveAndNotify_WhenAssigneeExists() throws IOException {
        IssueRequest request = new IssueRequest();
        request.setTitle("Bug Login");
        request.setDescription("Il sistema crasha all'avvio");
        request.setType(IssueType.BUG);
        request.setAssigneeId(10L);
```

```
MultipartFile file = mock(MultipartFile.class);
when(file.isEmpty()).thenReturn(false);
when(file.getBytes()).thenReturn(new byte[]{1, 2, 3});
when(file.getOriginalFilename()).thenReturn("log.txt");

User reporter = User.builder().id(1L).email("admin@test.com").build();
User assignee = User.builder().id(10L).email("dev@test.com").build();

when(currentUserService.getCurrentUser()).thenReturn(reporter);

when(userService.getUserById(10L)).thenReturn(Optional.of(assignee));

when(issueRepository.save(any(Issue.class)))
    .thenReturn(i -> i.getArguments()[0]);

IssueResponse response = issueService.createIssue(request, file);

verify(issueRepository).save(any(Issue.class));
verify(notificationService).notifyUser(eq(assignee),
    contains("Nuova Assegnazione"), anyString());
assertEquals("Bug Login", response.getTitle());
}

@Test
void createIssue_ShouldThrowException_WhenAssigneeNotFound() {
    IssueRequest request = new IssueRequest();
    request.setTitle("Bug Login");
    request.setAssigneeId(99L);

    User reporter = User.builder().id(1L).build();
    when(currentUserService.getCurrentUser()).thenReturn(reporter);

    when(userService.getUserById(99L)).thenReturn(Optional.empty());

    assertThrows(ResourceNotFoundException.class, () -> {
        issueService.createIssue(request, null);
    });

    verify(issueRepository, never()).save(any(Issue.class));
}

@Test
void createIssue_ShouldThrowFileStorageException_WhenFileReadFails() throws IOException {
    IssueRequest request = new IssueRequest();
    request.setTitle("Bug Login");

    MultipartFile file = mock(MultipartFile.class);
```

```

        when(file.isEmpty()).thenReturn(false);
        when(file.getBytes()).thenThrow(new IOException("Errore di lettura disco"));

        User reporter = User.builder().id(1L).build();
        when(currentUserService.getCurrentUser()).thenReturn(reporter);

        assertThrows(FileStorageException.class, () -> {
            issueService.createIssue(request, file);
        });

        verify(issueRepository, never()).save(any(Issue.class));
    }
}

```

Test 2: Delega agli Adapter (NotificationService)

Questo test valida il funzionamento dell'Adapter Pattern. Il `NotificationService` (Client) interagisce con l'interfaccia `NotificationAdapter` (Target). Si verifica che il messaggio venga delegato correttamente a tutti gli adattatori concreti iniettati nella lista.

```

@ExtendWith(MockitoExtension.class)
class NotificationServiceTest {

    @Mock private NotificationAdapter emailAdapter;
    @Mock private NotificationAdapter dbAdapter;

    @Mock private NotificationRepository notificationRepository;

    private NotificationService notificationService;

    @BeforeEach
    void setUp() {
        List<NotificationAdapter> adapters = Arrays.asList(emailAdapter, dbAdapter);

        notificationService = new NotificationService(adapters, notificationRepository);
    }

    @Test
    void notifyUser_ShouldDelegateToAllAdapters() {
        User user = new User();
        String subject = "Test Subject";
        String msg = "Hello World";

        notificationService.notifyUser(user, subject, msg);

        // Verifica chiamata su ENTRAMBI gli adapter
        verify(emailAdapter, times(1)).send(user, subject, msg);
        verify(dbAdapter, times(1)).send(user, subject, msg);
    }
}

```

```

@Test
void notifyUser_ShouldDoNothing_WhenRecipientIsNull() {
    notificationService.notifyUser(null, "Test Subject", "Hello");

    verify(emailAdapter, never()).send(any(), anyString(), anyString());
    verify(dbAdapter, never()).send(any(), anyString(), anyString());
}

@Test
void notifyUser_ShouldNotThrowException_WhenAdapterListIsEmpty() {
    NotificationService emptyService = new NotificationService(List.of(), notificationService);
    User user = new User();

    assertDoesNotThrow(() -> {
        emptyService.notifyUser(user, "Test Subject", "Hello");
    });
}
}

```

9.3 Strategie Black Box: Analisi del Dominio di Input

La progettazione dei casi di test è stata guidata dall’approccio **Black Box** (o *Specification-based testing*). I test sono stati derivati dall’analisi delle specifiche funzionali, trattando i componenti come ”scatole nere” e ignorando inizialmente la struttura interna del codice.

Per selezionare i dati di input in modo efficace ed evitare l’impossibile *Testing Esaustivo*, è stata applicata la tecnica del Partizionamento in Classi di Equivalenza.

9.3.1 Analisi del Test 1: Creazione Issue (IssueService)

Il metodo sotto test, `createIssue`, accetta due parametri complessi: un DTO `IssueRequest` e un oggetto binario `MultipartFile`. Il dominio di input è stato partizionato come segue:

- **Partizione 1: Parametro IssueRequest**

- *Classe Valida*: Un oggetto DTO contenente valori non nulli per i campi obbligatori (`title`, `description`) e un valore valido per l’Enum `IssueType` (es. `BUG`). Questa classe rappresenta i dati conformi ai vincoli di validazione.
- *Classe Invalida*: Oggetti che violano i vincoli di business o referenziali (es. ID di un assegnatario inesistente). Il sistema dovrebbe rifiutare questi input lanciando un’eccezione.

- **Partizione 2: Parametro MultipartFile**

- *Classe Valida*: Un file esistente con contenuto binario (dimensione in byte > 0) e metadati corretti.
- *Classe Invalida*: Riferimento nullo, file vuoto (0 byte) o file che genera errori di I/O durante la lettura.

Selezione dei Casi di Test (Each Choice Coverage) Per soddisfare pienamente il criterio *Each Choice Coverage* (ECC), che richiede di coprire ogni classe valida e invalida almeno una volta, è stata progettata una suite di test mirata:

- **Copertura Classi Valide (Happy Path):** Il test `createIssue_ShouldSaveAndNotify_WhenAss` copre la combinazione delle classi valide per entrambi i parametri, verificando il corretto salvataggio e l'invio della notifica.
- **Copertura Classi Invalide (Sad Path):**
 - Il test `createIssue_ShouldThrowException_WhenAssigneeNotFound` copre la partizione invalida della richiesta, garantendo che un assegnatario inesistente provochi il lancio di `ResourceNotFoundException` e interrompa il salvataggio.
 - Il test `createIssue_ShouldThrowFileStorageException_WhenFileReadFails` copre la classe invalida del parametro `MultipartFile` simulando un errore di I/O, assicurando che il sistema gestisca l'errore lanciando `FileStorageException`.

9.3.2 Analisi del Test 2: Delega agli Adapter (NotificationService)

Il secondo scenario di test si focalizza sulla validazione del **Pattern Adapter** e della logica di distribuzione dei messaggi. A differenza del test precedente, qui l'approccio è orientato alla **Verifica Comportamentale** (*Behavior Verification*): non si verifica il valore di ritorno (il metodo è `void`), ma ci si assicura che il componente sotto test interagisca correttamente con le sue dipendenze.

Partizionamento delle Dipendenze e degli Input In questo contesto, il partizionamento ha riguardato sia la configurazione del servizio (Dependency Injection) sia gli input ricevuti:

- *Classe Valida (Lista Popolata):* La lista degli adapter contiene $N \geq 1$ elementi. Il comportamento atteso è che il servizio iteri su tutti gli elementi e invochi il metodo `send`.
- *Classe Limite (Lista Vuota):* La lista è vuota (caso base del ciclo). Il sistema non deve generare eccezioni.
- *Input Invalido (Destinatario Nullo):* Il parametro `recipient` è nullo. Il sistema deve interrompere il flusso senza inoltrare chiamate.

Strategia di Testing con Mock Objects Per isolare il `NotificationService` dalle implementazioni concrete, sono stati utilizzati i Mock Objects. La suite implementata copre esaustivamente le partizioni individuate:

- Il test `notifyUser_ShouldDelegateToAllAdapters` verifica la *Classe Valida* configurando una lista con due mock distinti (`emailAdapter`, `dbAdapter`). Tramite le primitive di Mockito, si verifica l'esatta cardinalità delle chiamate (`times(1)`) e l'integrità dei dati trasmessi.
- Il test `notifyUser_ShouldNotThrowException_WhenAdapterListIsEmpty` garantisce la robustezza del sistema coprendo la *Classe Limite*.

- Il test `notifyUser_ShouldDoNothing.WhenRecipientIsNull` valida la *Classe Invalida*, certificando tramite la primitiva `never()` che nessuna dipendenza venga erroneamente invocata.

9.4 Strategie Strutturali e Isolation Testing

Parallelamente all'analisi funzionale, è stata adottata una strategia **White Box** (o *Structural-based testing*). Utilizzando la conoscenza della struttura interna del codice, i test sono stati progettati per massimizzare l'esecuzione delle istruzioni critiche e verificare le interazioni tra i componenti in isolamento.

9.4.1 Copertura Strutturale (Statement Coverage)

L'obiettivo primario è stato il raggiungimento di un'elevata **Statement Coverage** (Copertura delle Istruzioni). Analizzando il *Control Flow Graph* (CFG) dei metodi:

1. **Happy Path:** Il test principale su `createIssue` attraversa il ramo *True* delle condizioni principali (es. `assignee != null`), attivando la logica di notifica e forzando l'esecuzione dei blocchi di lettura stream (`file.getBytes()`).
2. **Gestione degli Errori:** L'aggiunta dei test sulle partizioni invalide ha permesso di attraversare sistematicamente i rami alternativi del CFG, coprendo le istruzioni di *throw* delle eccezioni e i blocchi `catch`, massimizzando così la copertura strutturale complessiva.

9.4.2 Isolation Testing con Mock Objects

Per garantire la ripetibilità e la velocità dei test, è stato applicato rigorosamente il principio dell'**Isolation Testing**. I componenti esterni (Database, File System, Server SMTP) sono stati sostituiti da **Test Doubles** di tipo *Mock*, implementati tramite il framework **Mockito**.

Verifica Comportamentale (Behavior Verification) Particolarmente critica per i metodi `void`, questa tecnica sposta l'attenzione dal controllo di stato al controllo delle interazioni. Utilizzando i metodi `verify()` e `never()` di Mockito, è stato possibile validare:

- **Esattezza delle Chiamate:** Si garantisce che i metodi delle dipendenze vengano invocati (o ignorati) in base al flusso esecutivo atteso.
- **Cardinalità:** Si certifica che le notifiche vengano inviate esattamente una volta per canale, o zero volte in caso di errore, prevenendo comportamenti anomali o spam.
- **Correttezza dei Parametri:** Si verifica che le entità passate ai servizi siano inoltrate inalterate.

9.4.3 Conclusioni sulla Strategia

La combinazione di tecniche Black Box (per la selezione degli input significativi, inclusi casi limite ed errori) e White Box (per la massimizzazione della copertura dei rami interni) ha permesso di ottenere una suite di test robusta e completa. L'uso dei Mock

ha infine disaccoppiato la logica di business dall'infrastruttura, rendendo i test immuni a malfunzionamenti di servizi esterni e fedeli ai principi del testing unitario.